

Comparing model checking and logical reasoning for real-time systems

Henning Dierks

Department of Computer Science, University of Oldenburg, Oldenburg, Germany

Abstract. We apply both model checking and logical reasoning to a real-time protocol for mutual exclusion. To this end we employ PLC-Automata, an abstract notion of programs for real-time systems. A logic-based semantics in terms of Duration Calculus is used to verify the correctness of the protocol by logical reasoning. An alternative but consistent operational semantics in terms of Timed Automata is used to verify the correctness by model checkers. Since model checking of the full model does not terminate in all cases within an acceptable time we examine abstractions and their influence on model-checking performance. We present two abstraction methods that can be applied successfully for the protocol presented.

Keywords: Real-time; Duration Calculus; Timed Automata; Verification; Model checking

1. Introduction

In recent years model checking of real-time systems has become popular because several tools have been developed implementing theoretical results (e.g. [ACD90]) about Timed Automata [AD94]. In this paper we examine the usage of these tools applied to an example which can be easily handled with Duration Calculus [ZHR91].

To this end we briefly introduce an abstract device of real-time systems called “PLC-Automata” [Die97, Die99]. These automata have been developed in the UniForM-project [KBPO⁺96] as a specification language for programs of “programmable logic controllers” (PLCs) which are simple computing devices controlling systems like production cells and traffic lights. PLC-Automata can be translated directly into source code for PLCs and have a Duration Calculus semantics describing the behaviour of a PLC executing the corresponding source code. In [DFMV98a] an operational semantics for PLC-Automata in terms of Timed Automata [AD94] has been developed and it was proven that both semantics are equivalent.

Due to the latter result we can now test the applicability of model checkers like Kronos [DOTY96, Yov97] and Uppaal [BLL⁺96, LPW97] by specifying a system using PLC-Automata. The operational semantics of this system in terms of Timed Automata can be fed into these tools and we try to prove a typical constraint, namely a mutual exclusion property. By simple reasoning with the Duration Calculus semantics we will see that this property is not fulfilled for a first version of the protocol. The model checkers are able to produce the same result within an acceptable time. After analysing the reason why the first version failed to ensure the mutual exclusion we present an improved protocol which is correct. The correctness can be established by logical reasoning in Duration Calculus quite easily. Model checking the improved version is partly more difficult. To overcome the problems we present abstraction methods for Timed Automata which can be used in our setting to make model checking feasible.

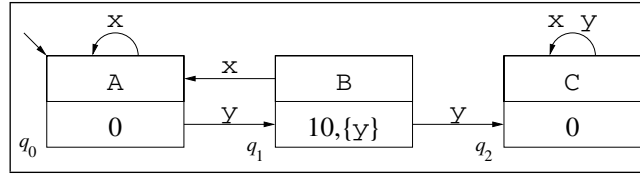


Fig. 1. An example of a PLC-Automaton

The logical reasoning employs Duration Calculus as a means to express temporal properties precisely and efficiently. It does not make use of known formal deduction rules because they are not applicable here. Nevertheless, all deductions presented in this paper are correct and one could easily establish appropriate sound deduction rules for them. Since this would lead to a very specialised set of rules this is omitted here.

2. PLC-Automata

In the UniForM-project [KBPO⁺96] an automaton-like device—called PLC-Automata—of polling real-time systems has been developed to enable a formal verification of PLC-programs. Basically, Programmable Logic Controllers (PLCs), the hardware aim of the project, can be viewed as simple computers with a special real-time operating system. They have features for making the design of time- and safety-critical systems easier:

- A PLC communicates with the environment via input and output channels. The environment may change the values of the inputs arbitrarily whereas the outputs are controlled by the PLC.
- They behave in a cyclic manner where every cycle consists of the following phases:
 - Poll all inputs and store the read values.
 - Compute the new values for the outputs.
 - Update all outputs.

The repeated execution of this cycle is managed by the operating system. The only part the programmer has to adapt is the computing phase.

- Depending on the program and on the number of inputs and outputs there is an upper time bound for a cycle. In the case of real-time systems we often require upper time bounds for reactions to certain events. The upper time bound for cycles is the main key to verify such properties.
- Convenient standardised libraries are given to simplify the handling of time.

In the formal definition of PLC-Automata below we consider the upper time bound for a polling cycle and the possibility of delay reactions of the system depending on state and input. Figure 1 gives an example of a PLC-Automaton. It shows an automaton with three states ($\{q_0, q_1, q_2\}$) and outputs $\{A, B, C\}$, that reacts to inputs of the alphabet $\{x, y\}$. Every state has two annotations in the graphical representation. The upper one denotes the output of the state; thus in state q_0 the output is A and in state q_2 the output is C. The lower annotation is either 0 or a pair (d, S) consisting of a real number $d > 0$ and a subset S of inputs.

A PLC-Automaton describes the behaviour of the system in the computation phase. The operational behaviour is similar to a finite state machine, i.e. depending on the polled input value the system changes both its state and its output. The behaviour is modified in only one case:

If

- the annotation of the current state is (d, S) and
- the polled input is in S and
- the current state does not hold longer than d seconds,

then no transition is executed.

The PLC-Automaton in Fig. 1 thus behaves as follows: It starts in state q_0 and remains there as long as it reads only the input x . The first time it reads y it changes to state q_1 . In q_1 the automaton reacts to the input x by changing the state back to q_0 independently of the time it stayed in state q_1 . It reacts to the input y by changing the state to q_2 provided that q_1 holds longer than 10 s. If this transition takes place the automaton enters q_2

and remains there forever. Hence, we know that the automaton changes its output to C when y holds a little bit longer than 10 s (the cycle time has to be considered). We formalise this graphic notation using an automaton-like structure extended by some components.

Definition 1 (PLC-Automaton) A tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, \varepsilon, S_t, S_e, \Omega, \omega)$ is a *PLC-Automaton* if

- Q is a nonempty, finite set of *states*,
- Σ is a nonempty, finite set of *inputs*,
- δ is a function of type $Q \times \Sigma \mapsto Q$ (*transition function*),
- $q_0 \in Q$ is the *initial state*,
- $\varepsilon > 0$ is the *upper bound* for a cycle,
- S_t is a function of type $Q \mapsto \mathbf{R}_{\geq 0}$ assigning to each state q a *delay time* how long the inputs contained in $S_e(q)$ should be ignored,
- S_e is a function of type $Q \mapsto \mathcal{P}(\Sigma)$ assigning to each state a set of *delayed inputs*,¹
- Ω is a nonempty, finite set of *outputs*, and
- ω is a function of type $Q \mapsto \Omega$ (*output function*)

The components Q , Σ , δ , and q_0 have the same meaning as in usual finite state automata. The additional components are needed to model a polling behaviour and to enrich the language for dealing with real-time aspects. The ε represents the upper time bound for a polling cycle and enables us to model this cycle in the semantics. The delay function S_t and S_e represent the annotations of the states. In the case of $S_t(q) = 0$ no delay time is given and the value $S_e(q)$ is arbitrary. If the delay time $S_t(q)$ is greater than 0 the set $S_e(q)$ denotes the set of inputs for which the delay time is valid.

Note that PLC-Automata are implementable. In [Die99] a translation of PLC-Automata into source code for PLCs is given. Recently also a translation into code for Lego Mindstorms has been developed. The logical relationship between the execution of the code by the real-world hardware and the semantics given in the rest of this paper is *refinement*. In other words: the real-world implementation cannot show a behaviour that is not covered by the formal semantics. Due to the lack of a formal semantics for the implementation languages we cannot establish this claim formally.

3. The Duration Calculus

In the following sections we shall demonstrate the various construction techniques of our approach. In our case study the specification language will be the Duration Calculus [ZHR91, Zho93, HZ97] (DC for short). It is a real-time interval temporal logic extending earlier work on discrete interval temporal logic of [Mos85].

A formal description of a real-time system using DC starts by choosing a number of time-dependent state variables (“observable”) obs of a certain type. An interpretation I assigns to each state variable a function $obs_I : \text{Time} \mapsto D$ where Time is the time domain, here the non-negative reals, and D is the type of obs . If D is finite, these functions obs_I are required to be *finitely variable*, which means that any interval $[b, e] \subseteq \text{Time}$ can be divided into finitely many subintervals such that obs_I is constant on the open subintervals.

State assertions P are obtained by applying propositional connectives to elementary assertions of the form $obs = v$ (v for short if obs is clear) for a $v \in D$. For a given interpretation I state assertions denote functions $P_I : \text{Time} \mapsto \{0, 1\}$.

Duration terms are of type real and their values depend on a given time interval $[b, e]$. The simplest duration term is the symbol ℓ denoting the length $e - b$ of $[b, e]$. For each state assertion P there is a duration term $\int P$ measuring the duration of P , i.e. the accumulated time P holds in the given interval. Semantically, $\int P$ denotes $\int_b^e P_I(t) dt$ on the interval $[b, e]$.

Duration formulas are built from arithmetical relations applied to duration terms, the special symbols **true** and **false**, and other terms of type real, and they are closed under propositional connectives and quantification over rigid variables. Their truth values depend on a given interval. We use F for a typical duration formula. **true** and **false** evaluate to true resp. false on every given interval. Further basic duration formulas are:

Relation over Durations: For example, $\int P = k$ expresses that the *duration* of the state assertion P in $[b, e]$ is k .

¹ If $S_t(q) = 0$ the set $S_e(q)$ can be arbitrarily chosen. The single 0 represents this in the graphical notation (cf. Fig. 1).

Chop: The composite duration formula $F_1; F_2$ (read as F_1 chop F_2) holds in $[b, e]$ if this interval can be divided into an initial subinterval $[b, m]$ where F_1 holds and a final subinterval $[m, e]$ where F_2 holds.

Besides this basic syntax various abbreviations are used:

$$\begin{aligned} \text{point interval: } \quad \square &\stackrel{\text{df}}{=} \ell = 0 \\ \text{everywhere: } \quad \lceil P \rceil &\stackrel{\text{df}}{=} \int P = \ell \wedge \ell > 0 \\ \text{somewhere: } \quad \diamond F &\stackrel{\text{df}}{=} \text{true}; F; \text{true} \\ \text{always: } \quad \square F &\stackrel{\text{df}}{=} \neg \diamond \neg F \end{aligned}$$

Since phases are often assigned a duration requirement the following abbreviations are useful:

$$\begin{aligned} F^t &\stackrel{\text{df}}{=} (F \wedge \ell = t) \\ F^{\sim t} &\stackrel{\text{df}}{=} (F \wedge \ell \sim t) \text{ with } \sim \in \{<, \leq, >, \geq\} \end{aligned}$$

A duration formula F holds in an interpretation I if F evaluates to true in I and every interval of the form $[0, t]$ with $t \in \text{Time}$. If convenient, we use F to describe a set of interpretations, namely all interpretations in which F holds.

The following so-called *standard forms* are useful to describe dynamic behaviour:

$$\begin{aligned} \text{followed-by: } \quad F &\longrightarrow \lceil P \rceil \stackrel{\text{df}}{=} \square \neg (F; \lceil \neg P \rceil) \\ \text{timed leads-to: } \quad F &\xrightarrow{t} \lceil P \rceil \stackrel{\text{df}}{=} (F \wedge \ell = t) \longrightarrow \lceil P \rceil \\ \text{timed up-to: } \quad F &\xrightarrow{\leq t} \lceil P \rceil \stackrel{\text{df}}{=} (F \wedge \ell \leq t) \longrightarrow \lceil P \rceil \end{aligned}$$

To avoid parentheses the following precedence rules are used:

1. $\int$
2. real operators
3. real predicates
4. $\neg, \square, \diamond$
5. $;$
6. $\wedge, \vee$
7. $\implies, \longrightarrow, \xrightarrow{\leq t}, \xrightarrow{t}$
8. quantification

4. The logical semantics

Let $\mathcal{A} = (\mathcal{Q}, \Sigma, \delta, q_0, \varepsilon, S_e, S_t, \Omega, \omega)$ be a PLC-Automaton. The Duration Calculus semantics $\llbracket \mathcal{A} \rrbracket_{\text{DC}}$ of $\mathcal{A}$ is given by the conjunction of the predicates (1)–(11) regarding the state variables

$$\begin{aligned} \text{state :Time} &\longmapsto \mathcal{Q} \\ \text{input :Time} &\longmapsto \Sigma \\ \text{output :Time} &\longmapsto \Omega. \end{aligned}$$

First of all, the starting of the automaton in the proper initial state is expressed by:

$$\square \vee \lceil q_0 \rceil; \text{true}. \tag{1}$$

Note that $\lceil q_0 \rceil$ is an abbreviation of $\lceil \text{state} = q_0 \rceil$. Next, we want to describe the behaviour of the automaton in state q . The cyclic behaviour of PLCs has to be reflected in the semantics to achieve a realistic modelling. One question the semantics should answer is: When a state q is entered, what kind of input can influence the behaviour of the PLC? The answer to this question is:

- only the inputs after entering q and
- only the inputs during the last cycle-time ε .

This is expressed by the following predicates where A ranges over all *sets of inputs* with $\emptyset \neq A \subseteq \Sigma$:

$$\lceil \neg q \rceil ; \lceil q \wedge A \rceil \longrightarrow \lceil q \vee \delta(q, A) \rceil \quad (2)$$

$$\lceil q \wedge A \rceil \xrightarrow{\varepsilon} \lceil q \vee \delta(q, A) \rceil \quad (3)$$

Note that we use A in the formulas as an abbreviation for $\text{input} \in A$ resp. $\delta(q, A)$ for $\text{state} \in \{\delta(q, a) \mid a \in A\}$. The statement (2) formalises the fact that after a change of the automaton's state to q only the set of inputs A that is valid after the change can have an effect on the behaviour in the future. The statement (3) represents the formalisation of the cyclic behaviour of PLCs. A PLC reacts only to inputs during the last cycle. Preceding inputs are forgotten and cannot influence the PLC anymore. The quantification over all nonempty subsets of the input alphabet was motivated by the behaviour of the PLCs. The more we know about the inputs during the last cycle the more we know about the actions of the PLC.

For states without a stability requirement we expect a change to $\delta(q, A)$ or more generally we expect that the automaton reacts accordingly in at most 2ε seconds. For states with a stability requirement we expect this behaviour after the required period of time. This leads us to additional statements in the semantics:

$$S_i(q) = 0 \implies \lceil q \wedge A \rceil \xrightarrow{2\varepsilon} \lceil \delta(q, A) \rceil \quad (4)$$

$$S_i(q) > 0 \implies \lceil q \rceil ; \lceil q \wedge A \rceil^{2\varepsilon} \xrightarrow{2\varepsilon + S_i(q)} \lceil \delta(q, A) \rceil \quad (5)$$

$$S_i(q) = 0 \wedge q \notin \delta(q, A) \implies \lceil \neg q \rceil ; \lceil q \wedge A \rceil^\varepsilon \longrightarrow \lceil \neg q \rceil \quad (6)$$

Statement (4) says that after at most 2ε seconds the automaton reacts to the input accordingly if there is no stability required for q . Note that ε seconds are needed to assure that the PLC reads at least once and ε seconds are needed to react to this input in the worst case. Formula (5) states this behaviour after $S_i(q)$ seconds: if $S_i(q)$ seconds have elapsed the automaton reacts to inputs in at most 2ε seconds. In the case that we know that the automaton has just changed the state then we want to be able to exploit the information that within the next ε seconds another reaction to the inputs in A has to occur. This is formalised by (6).

Next we want to describe the automaton's behaviour if it is in a state q where a stability is required and the $S_i(q)$ seconds have not elapsed. Then we want to hold this state provided that during this phase only inputs in $S_e(q)$ are read. That means inputs in $S_e(q)$ cannot cause a change of state during the first $S_i(q)$ seconds:

$$S_i(q) > 0 \implies \lceil \neg q \rceil ; \lceil q \wedge A \rceil \xrightarrow{\leq S_i(q)} \lceil q \vee \delta(q, A \setminus S_e(q)) \rceil \quad (7)$$

However, we have to take into account the cyclic behaviour of the hardware again. In particular, we should require that if q is left during the stability phase then there has to be an input not contained in $S_e(q)$ at most ε seconds ago:

$$S_i(q) > 0 \implies \lceil \neg q \rceil ; \lceil q \rceil ; \lceil q \wedge A \rceil^\varepsilon \xrightarrow{\leq S_i(q)} \lceil q \vee \delta(q, A \setminus S_e(q)) \rceil \quad (8)$$

Furthermore, we know that the automaton reacts according to the input if there is a set A that is valid for the last 2ε seconds and disjoint from $S_e(q)$:

$$S_i(q) > 0 \wedge A \cap S_e(q) = \emptyset \wedge q \notin \delta(q, A) \implies \lceil q \wedge A \rceil \xrightarrow{2\varepsilon} \lceil \delta(q, A) \rceil \quad (9)$$

$$S_i(q) > 0 \wedge A \cap S_e(q) = \emptyset \wedge q \notin \delta(q, A) \implies \lceil \neg q \rceil ; \lceil q \wedge A \rceil^\varepsilon \longrightarrow \lceil \neg q \rceil \quad (10)$$

Formula (10) corresponds to (6). Note that (2), (6), (7), (8), and (10) require a change from $\lceil \neg q \rceil$ to $\lceil q \rceil$ to restrict the possible behaviour. But for the initial state there is no change and therefore the assertions are not applicable in this case. This can be expressed by five corresponding assertions suitable for the initial state, which for brevity are omitted here.

Finally, the relation between the state variables **state** and **output** is established by

$$\Box(\lceil q \rceil \implies \lceil \omega(q) \rceil) \quad (11)$$

The latter formula allows *no* time delay between an entry of a new state and the issuing of the corresponding output. This is justified by the black box view on the PLC. The semantics does not intend to describe the internal behaviour of the PLC. It describes the external observable behaviour of the system. For example, it does not use a state variable that describes the *polled* input value. From the perspective of the external observer the state variables **state** and **output** change their values at the same time points.

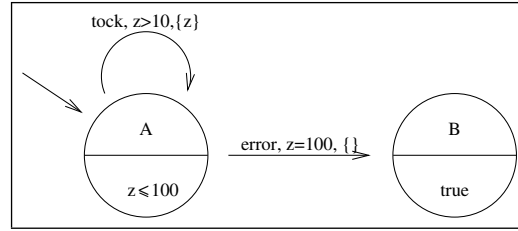


Fig. 2. A Timed Automaton

5. Timed Automata

Timed Automata [AD90, ACD90, ACD93, AD94] are the predominant model for real-time systems with continuous time. Basically, they are simple finite state automata extended by clocks. These clocks are variables with domain $\mathbf{R}_{\geq 0}$ which increase uniformly with time. Transitions of the automaton carry guards consisting of boolean expressions for the comparisons between a clock value (or the difference of two clock values) and a constant. If a transition is taken then a subset of clocks can be reset. Moreover the states of the automaton can be labelled with a guard of the same type as the transitions. These guards represent invariants restricting the admissible clock values.

In Fig. 2 a timed automaton is given that starts its computation in the location with the proposition A and holds this state as long as it can perform the transition with label *tock*. This transition can only be taken when the z -clock has a value greater than 10. As long as $z \leq 10$ holds, no transition is allowed. If the *tock*-transition is taken, the z -clock will be reset to 0. If z reaches the value 100 the automaton may take the transition with label *error* to enter a state with proposition B . Due to the fact that the A -state has the invariant $z \leq 100$ it has to take one of the transitions if z reaches 100.

A system of Timed Automata in parallel composition uses labels (*tock* and *error* in Fig. 2) for synchronisation. It is assumed that transitions happen instantaneously, which means that they do not consume time.

Although the basic concepts are very similar, various definitions of syntax and semantics can be found in the literature [ACD90, NSY92, AD94, HNSY94, MP95, MY96]. Here we use a variant of timed automata that is defined in [MY96]:

Definition 2 (Timed Automaton) A *timed automaton* $\mathcal{T}$ is a tuple $(S, \mathcal{X}, \mathcal{L}, \mathcal{E}, \mathcal{IV}, \mathcal{P}, \mu, S_0)$ where:

- S is a finite set of *locations*,
- $\mathcal{X}$ is a finite set of real-valued variables called *clocks* whose values increase uniformly with time,
- $\mathcal{L}$ is a finite set of *labels*.
- $\mathcal{E}$ is a finite set of *edges* of the form $e = (s, L, \phi, \rho, s')$, or alternatively written as $s \xrightarrow{L, \phi, \rho} s'$, where $s, s' \in S$, $L \in \mathcal{L}$, ϕ is a *clock constraint*, generated by the grammar

$$\phi ::= x + c \leq d \mid c \leq x + d \mid x + c \leq y + d \mid \neg \phi \mid \phi_1 \wedge \phi_2$$

with $x, y \in \mathcal{X}$ and $c, d \in \mathbf{R}$, and $\rho \subseteq \mathcal{X}$ is the set of clocks which are to be reset to 0 by the transition,

- $\mathcal{IV}$ assigns to each location a clock constraint that serves as an *invariant* within the location,
- $\mathcal{P}$ is a finite set of atomic propositions,
- μ is a labelling of the locations with a set of atomic propositions over $\mathcal{P}$,
- $S_0 \subseteq S$ is the set of *initial locations*.

Usually only natural numbers are allowed as constants in the clock constraints, but in order to associate a timed automaton to each PLC-Automaton our definition allows for real-valued constants. The price we have to pay is that we cannot model-check this kind of timed automata. However, as long as the PLC-Automaton uses only discrete delays and a discrete cycle time, the corresponding timed automaton semantics uses only discrete time constants, too.

Definition 3 (Run of a timed automaton) A run of $\mathcal{T}$ is an infinite sequence $r = ((s_i, v_i, t_i))_{i \in \mathbb{N}_0}$ where, for each $i \in \mathbb{N}_0$,

- $s_i \in \mathcal{S}$ is a location,
- $v_i \in \mathcal{X} \mapsto \mathbf{R}_{\geq 0}$ is a *valuation* of the clocks,
- $t_i \in \mathbf{R}_{\geq 0}$ is a time stamp,

and r satisfies the following properties:

- the initial location is contained in S_0 : $s_0 \in S_0$,
- initially all the clocks have value 0: $\forall x \in \mathcal{X} : v_0(x) = 0$,
- time starts at 0: $t_0 = 0$,
- the sequence of time stamps is monotonic and diverging: $t_i \leq t_{i+1}$, for all $i \in \mathbb{N}_0$, and $\lim_{i \rightarrow \infty} t_i = \infty$,
- for all $i \in \mathbb{N}_0$ the invariant $\mathcal{IV}(s_i)$ is fulfilled during $[t_i, t_{i+1}]$:

$$\forall 0 \leq t \leq t_{i+1} - t_i : \mathcal{IV}(s_i)(v_i + t)$$

with $(v_i + t)(x) \stackrel{\text{df}}{=} v_i(x) + t$ for all $x \in \mathcal{X}$ and $\mathcal{IV}(s)(v)$ denoting the evaluation of the constraint $\mathcal{IV}(s)$ at valuation v ,

- for all $i \in \mathbb{N}_0$ there is an edge $e = (s_i, L, \phi, \rho, s_{i+1})$ such that
 - clock constraint ϕ holds at time t_{i+1} : $\phi(v_i + t_{i+1} - t_i)$, and
 - valuation v_{i+1} is updated according to ρ :

$$\forall x \in \mathcal{X} : v_{i+1}(x) = \begin{cases} 0 & \text{if } x \in \rho \\ v_i(x) + t_{i+1} - t_i & \text{if } x \notin \rho \end{cases}$$

By $\mathcal{R}(\mathcal{T})$ we denote the set of runs of a timed automaton $\mathcal{T}$.

6. The model-check semantics

In the following we give an operational semantics of a PLC-Automaton in terms of a set of timed traces accepted by a Timed Automaton. In [DFMV98a] it was proven that the following semantics is stronger than the Duration Calculus semantics above (and equivalent to a slight extension of the semantics in Sect. 4).

Definition 4 (Timed Automaton semantics) Let $\mathcal{A}$ be a PLC-Automaton with $\mathcal{A} = (Q, \Sigma, \delta, q_0, \varepsilon, S_t, S_e, \Omega, \omega)$. We define $\mathcal{T}(\mathcal{A}) \stackrel{\text{df}}{=} (\mathcal{S}, \mathcal{X}, \mathcal{L}, \mathcal{E}, \mathcal{I}, \mathcal{P}, \mu, S_0)$ with

- $\mathcal{S} \stackrel{\text{df}}{=} \{0, 1, 2, 3\} \times \Sigma \times \Sigma \times Q$ as locations,
- $\mathcal{X} \stackrel{\text{df}}{=} \{x, y, z\}$ as clocks,
- $\mathcal{L} \stackrel{\text{df}}{=} \Sigma \cup \{\text{poll}, \text{test}, \text{tick}\}$ as labels,
- The set of transitions $\mathcal{E}$ is given in Fig. 3 where $i \in \{0, 1, 2, 3\}$, $a, b, c \in \Sigma$, and $q \in Q$,
- $\mathcal{IV}(s) \stackrel{\text{df}}{=} z \leq \varepsilon$ as invariant for each location $s \in \mathcal{S}$,
- $\mathcal{P} = \Sigma \cup Q \cup \Omega$ is the set of propositions,
- $\mu(i, a, b, q) \stackrel{\text{df}}{=} a \wedge q \wedge \omega(q)$ as propositions for each location $(i, a, b, q) \in \mathcal{S}$, and
- $S_0 \stackrel{\text{df}}{=} \{(0, a, b, q_0) | a, b \in \Sigma\}$ as set of initial locations.

The set of locations² of $\mathcal{T}(\mathcal{A})$ (refer Def. 4) consists of four dimensions. The first part (“program counter”) with range $\{0, 1, 2, 3\}$ describes the internal status of the polling system. “0” denotes the first part of the cycle. The polling has not occurred yet. “1” denotes that the polling has happened in the current cycle. The check whether to react has not occurred yet. “2” denotes that polling and testing have happened. The system decided not to

² Note that “locations” refers to the Timed Automaton and “states” to the PLC-Automaton.

$(i, a, b, q) \xrightarrow{c, \text{true}, \{x\}} (i, c, b, q)$	if $c \neq a$	(12)
$(0, a, b, q) \xrightarrow{\text{poll}, 0 < x \wedge 0 < z, \emptyset} (1, a, a, q)$		(13)
$(1, a, b, q) \xrightarrow{\text{test}, y \leq S_t(q), \emptyset} (2, a, b, q)$	if $S_t(q) > 0 \wedge b \in S_e(q)$	(14)
$(1, a, b, q) \xrightarrow{\text{test}, y > S_t(q), \emptyset} (3, a, b, q)$	if $S_t(q) > 0 \wedge b \in S_e(q)$	(15)
$(1, a, b, q) \xrightarrow{\text{test}, \text{true}, \emptyset} (3, a, b, q)$	if $S_t(q) = 0 \vee b \notin S_e(q)$	(16)
$(2, a, b, q) \xrightarrow{\text{tick}, \text{true}, \{z\}} (0, a, b, q)$		(17)
$(3, a, b, q) \xrightarrow{\text{tick}, \text{true}, \{z\}} (0, a, b, \delta(q, b))$	if $q = \delta(q, b)$	(18)
$(3, a, b, q) \xrightarrow{\text{tick}, \text{true}, \{y, z\}} (0, a, b, \delta(q, b))$	if $q \neq \delta(q, b)$	(19)

Fig. 3. The transitions of the TA-semantics

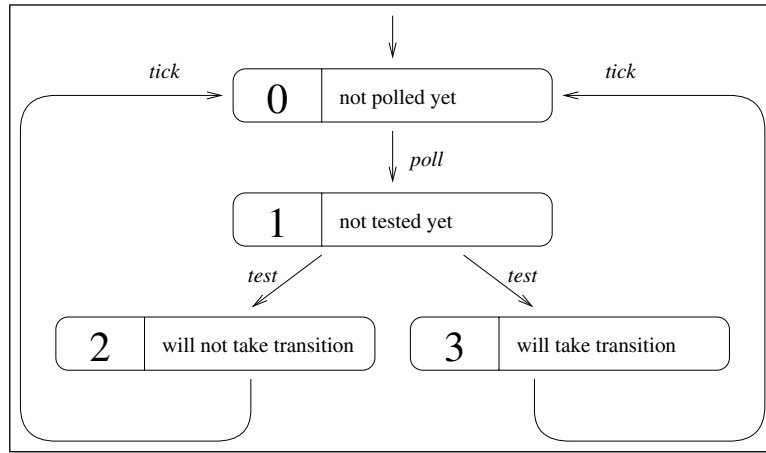


Fig. 4. The program counter

react to the input. “3” denotes that polling and testing have happened. The system decided to react to the input (cf. Fig. 4).

The second component of the locations denotes the latest input event while the third component contains the latest polled input. The last component represents the current state of the PLC-Automaton. There are three clocks in use: Clock x measures how long the latest input is valid, clock y measures how long the current state is valid, and clock z measures the time of the current cycle. Transitions that change the first component of the locations are labelled with *poll*, *test*, and *tick*. The remaining transitions (12) are labelled with inputs and are not restricted anyhow. They change the second component which represents the latest input-event. The third component describes the input which is polled by the system. The polling has to happen after an amount of time since the beginning of the cycle. To this end the clock z is used. This clock denotes the elapsed time for the current cycle. Hence, the polling transition (13) is labelled with the condition $z > 0$. Furthermore, it is not allowed to poll an input at the same time point where it gets valid. Hence, we introduced the clock x denoting the time since the last input is valid and restricted the *poll*-event with $x > 0$. Otherwise the system could react to input that was valid only for a point of time which has to be avoided in order to keep the consistency with the DC-semantics. After the polling the testing has to occur (14)–(16). These transitions reflect the decision of the system whether to react to the polled b input or not. It depends on the definitions of S_e , S_t , and the value of the y -clock which denotes the time how long the current *state* q is valid. It can only decide to ignore the input when $b \in S_e(q)$ and $S_t(q) > 0$ are true (14) and moreover the delay time has not elapsed: $y \leq S_t(q)$. Finally, the *tick*-events finish the cycle (17)–(19). Depending on the previous decision by the *test*-event the state may change or not. All necessary clocks are reset. Due to the invariants $z \leq \varepsilon$ for all locations we know that a cycle consisting of a *poll*-, a *test*-, and a *tick*-event has to happen within ε s because only the *tick*-events reset z .

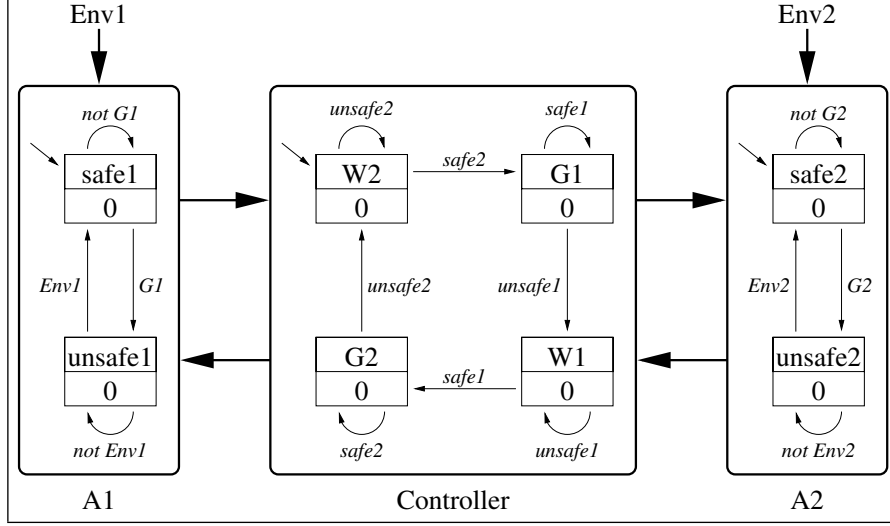


Fig. 5. The system with three PLC-Automata

7. A “Protocol”

Now we consider a system of PLC-Automata which works in parallel. Two automata (“A1”, “A2”) have a state which is “unsafe”. The automaton “Controller” is used to assure the mutual exclusion of both unsafe states (Fig. 5). In the Duration Calculus semantics the parallel composition is simply the conjunction of the semantics of the three automata in the system in addition with the following linking formulas:

$$\Box(\Box \vee [\text{input}_{A1}^1 = \text{output}_{Ctrl}]) \quad \Box(\Box \vee [\text{output}_{A1} = \text{input}_{Ctrl}^1]) \quad (20)$$

$$\Box(\Box \vee [\text{input}_{A2}^1 = \text{output}_{Ctrl}]) \quad \Box(\Box \vee [\text{output}_{A2} = \text{input}_{Ctrl}^2]) \quad (21)$$

These formulas say that the output of Controller is the input of A1 and A2. The outputs of A1 and A2 are inputs of Controller. Note that we defined the PLC-Automaton only for one input. It is obvious how to extend this definition to cope with multiple inputs using the Cartesian product.

The influence of the environment is modelled by the inputs Env1 and Env2 of Boolean type. In Fig. 5 the transitions are labelled with predicates over the inputs instead of input-elements to enhance readability. The Controller waits in state W_i (“Wait”, $i \in \{1, 2\}$) as long as A_i is unsafe. If the Controller recognises that A_i is safe, it changes to $G(3-i)$ (“Grant”) which allows $A(3-i)$ to enter its unsafe state. The grant is given until the Controller detects that $A(3-i)$ has entered the unsafe state. The mutual exclusion shall be guaranteed by the W_i -states. We assume that these PLC-Automata are equipped with the upper cycle bounds $\varepsilon_1 = \varepsilon_2 = \varepsilon_C$.

Now we are interested whether we can prove the mutual exclusion of both unsafe-states. Due to the existence of two semantics we can compare the logical approach with the model-checking approach. For the model checking of the system we used the model checker Kronos [DOTY96] (Version 2.4.4, August 2000) and UPPAAL [BLL⁺96] (Version 3.1.64, June 2001).

7.1. Using Duration Calculus

First we try to prove the mutual exclusion property of the protocol by reasoning in Duration Calculus. This means, that we have to prove this property:³

$$[[A1]]_{DC} \wedge [[Ctrl]]_{DC} \wedge [[A2]]_{DC} \wedge (20) \wedge (21) \implies \neg \Diamond([u_1 \wedge u_2])$$

³ Note the mutual exclusion property ($\neg \Diamond([u_1 \wedge u_2])$) only uses two state variables whereas the semantics of the automata and the linking formulas use some more. There are some extensions of DC that allow for quantification over state variables. This would enable us to hide the additional state variables by quantifiers. However, in this simple case it suffices to conceive the DC formula $\neg \Diamond([u_1 \wedge u_2])$ as a specification of all state variables used by the semantics.

Note that we abbreviate *unsafe_i* and *safe_i* by u_i and s_i , respectively. To prove this we try to derive a contradiction from

$$\Diamond([u_1 \wedge u_2])$$

by the assumptions. In the first step we exploit the information about the initial states (1). Both automata A1 and A2 start in the safe state, hence we have to observe a state change from a safe situation (i.e. one automaton is in the safe state) to the unsafe situation (both automata are unsafe). Without loss of generality we assume that the second automaton executes the incorrect transition:

$$\{\text{W.l.o.g., (1)}\} \implies \text{true}; [s_2]; [u_1 \wedge u_2]; \text{true}$$

By the definition of A2 we know that the transition from safe2 to unsafe2 requires that a grant is given less than ε_2 seconds ago:

$$\{(2), (3)\} \implies \text{true}; \left(\begin{array}{l} [G2]; \ell < \varepsilon_2 \\ \wedge \text{true}; [s_2] \end{array} \right); [u_1 \wedge u_2]; \text{true}$$

By the definition of the controller's transitions we know that the controller has to visit the states W2, G1, and W1 in this order before it may enter G2:

$$\{(1), (2)\} \implies \text{true}; [W2]; [G1]; [W1]; \left(\begin{array}{l} [G2]; \ell < \varepsilon_2 \\ \wedge \text{true}; [s_2] \end{array} \right); [u_1 \wedge u_2]; \text{true}$$

Now we consider the behaviour of A1 during the $[W1]$ -phase. Due to (2) and (3) for A1 we know that transitions from s_1 to u_1 are only possible if a grant was given while s_1 and less than ε_1 seconds ago. With similar arguments for the controller we can conclude that the transition from W1 to G2 requires a $[s_1]$ -phase during the $[W1]$ -phase less than ε_c time units before the transition happens. Hence, we can categorise the behaviour of A1 during the $[W1]$ -phase in four cases:

$$\{(2), (3)\} \implies \text{true}; [W2]; [G1]; \left(\begin{array}{l} [W1] \wedge \\ \left(\begin{array}{l} [u_1]; [s_1] \\ \vee [s_1]^{<\varepsilon_1}; [u_1]; [s_1] \\ \vee [s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_c} \\ \vee [s_1] \geq \varepsilon_1 \end{array} \right) \end{array} \right); \left(\begin{array}{l} [G2]; \ell < \varepsilon_2 \\ \wedge \text{true}; [s_2] \end{array} \right); [u_1 \wedge u_2]; \text{true}$$

At the last line we can stop the proof attempt. The reason is that the case distinction discovers a case $([s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_c})$ where our system could fail. In all other cases we could derive the desired contradiction because due to (2) and (3) for A1 the state is not allowed to change to u_1 during the $[G2]$ -phase. But in the case $[s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_c}$ we have lost, because the Controller has left W1 too early believing that A1 is in the safe state. From above it is easy to derive a complete trace how the system can violate the mutual exclusion property (cf. Fig. 6 where the polling events are sketched by arrows).

Thus, Duration Calculus helped us to discover the error in our protocol because it gave us an appropriate way to reason about phases and their durations.

7.2. Using model checkers

In order to apply model checkers for Timed Automata we have to translate the specification of the protocol into in the syntaxes of the tools Kronos and Uppaal. The tool Moby/PLC [DT98, TD98] for PLC-Automata offers compiling functions for arbitrary systems of PLC-Automata such that the translation can be done mechanically.

To apply Kronos we have to translate each of the three automata separately into the Kronos-format "timed graphs". The resulting timed graphs had the following dimensions given in Table 1. In addition we have to add two small automata to model the environment. They are designed such that Env1 and Env2 may change arbitrarily. The size of the product automaton computed by Kronos is also given in the table. Kronos is able to model-check whether a system of Timed Automata satisfy a formula given in the logic TCTL.⁴ The TCTL-formula that expresses mutual exclusion is

⁴ An introduction of TCTL is omitted because (22) is the only TCTL-formula we need in this paper. See [ACD90] for a definition of TCTL.

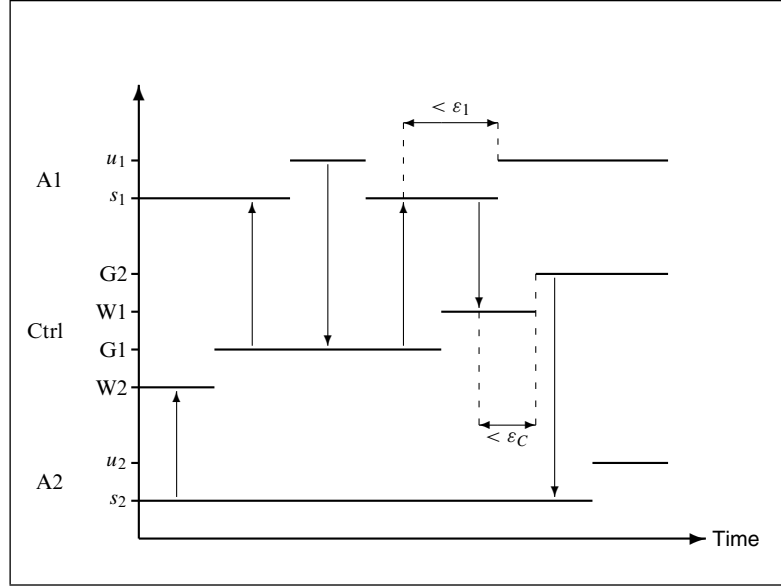


Fig. 6. A counterexample

Table 1.

	locations	transitions	integers	clocks
Kronos				
A1	49	342	–	2
Controller	65	324	–	2
A2	49	342	–	2
Env. for A1	3	4	–	–
Env. for A2	3	4	–	–
product	1681	8404	–	6
Uppaal				
A1	2	16	4	2
Controller	4	24	4	2
A2	2	16	4	2
Env. for A1	1	2	1	–
Env. for A2	1	2	1	–

$$\neg \exists \text{ true } \mathcal{U}_{\geq 0} (u_1 \wedge u_2). \quad (22)$$

The formula can be read as follows: There exists no run of the system in which a state occurs that fulfils $(u_1 \wedge u_2)$.

Model checking the product automaton if the mutual exclusion property is fulfilled yields the result “false” within approx. 16 s using an UltraSPARC 1 machine with a depth-first strategy and within about 2.5 min on the same machine with breadth-first. If Kronos is applied with the backward analysis option we get the same answer within 3 s. The drawback of the backward analysis option is that no counterexamples are provided and that makes it difficult to get confidence into the model of the system and to understand why the verification failed.

The model checker Uppaal is able to accept integer-variables. Therefore, we use an alternative representation of the Timed Automaton semantics for PLC-Automata. The integers are used to represent the program counter, the polled inputs, and the outputs. This makes the description of the system in Fig. 5 acceptably small (see Table 1). Uppaal can check a system of Timed Automata against reachability properties like (22) only whereas a forward analysis algorithm is used. With the same machine as above Uppaal discovers a counterexample in approx. 9 s.⁵ Both tools produced a counterexample which corresponds to the one presented in Fig. 6.

⁵ With the depth-first option. In case of breadth-first search it takes approx. 10 min.

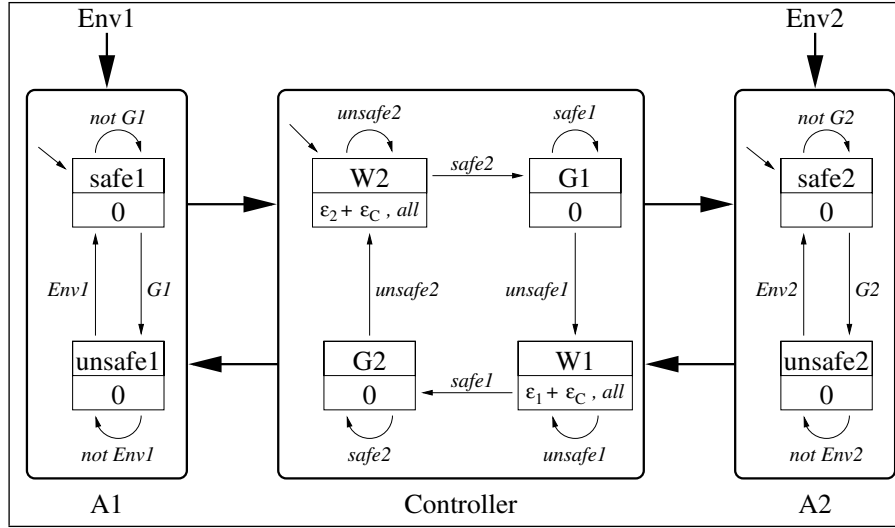


Fig. 7. The system with an improved controller

8. An improved protocol

We can derive an improved version of our protocol from the counterexample given in Fig. 6. The idea of the improvement is that we avoid the only situation in which the previous protocol can fail. This can be done by using delays to ensure that the states of A1 and A2 are checked over a certain time period. This is possible by introducing delays for the waiting states (cf. Fig. 7). The delay of $\varepsilon_i + \varepsilon_C$ seconds for state W_i is valid for *all* inputs. Again we try to verify the protocol with logical reasoning and model checking.

We can apply the same arguments as in Sect. 7.1 to derive the formula

$$\dots \{(2), (3)\} \Rightarrow \text{true}; [W2]; [G1]; \left(\begin{array}{c} [W1] \wedge \\ \left(\begin{array}{c} [u_1]; [s_1] \\ \vee [s_1]^{<\varepsilon_1}; [u_1]; [s_1] \\ \vee [s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_C} \\ \vee [s_1] \geq \varepsilon_1 \end{array} \right) \end{array} \right); \\ \left(\begin{array}{c} [G2]; \ell < \varepsilon_2 \\ \wedge \text{true}; [s_2] \end{array} \right); [u_1 \wedge u_2]; \text{true}$$

As before (2) and (3) can be used to get a contradiction for all cases except for the subformula $[s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_C}$:

$$\{(2), (3)\} \Rightarrow \text{true}; [W2]; [G1]; ([W1] \wedge [s_1]^{<\varepsilon_1}; [u_1]^{<\varepsilon_C}); \quad (23) \\ \left(\begin{array}{c} [G2]; \ell < \varepsilon_2 \\ \wedge \text{true}; [s_2] \end{array} \right); [u_1 \wedge u_2]; \text{true}$$

Formula (7) with $q \stackrel{\text{df}}{=} W1$ and $A \stackrel{\text{df}}{=} \text{all}$ can be written as

$$[\neg W1]; [W1] \xrightarrow{\leq \varepsilon_1 + \varepsilon_C} [W1]$$

which says that state W1 holds at least $\varepsilon_1 + \varepsilon_C$ seconds and this contradicts (23). Hence, we proved mutual exclusion for the improved protocol.

The model-checking results for the improved protocol are as follows: When we use the backward analysis option of Kronos we get the correct result within 7 s. Forward analysis fails: Kronos exceeded the memory limitation of 700 MB (> 1 h). Uppaal was able to produce the correct result but it took approx. 4 h.

9. Abstractions

The partially disappointing model-checking results in Sect. 8 call for ways of improvements. The forward analysis failed to verify the mutual exclusion or it took a lot of time. Only the backward analysis gave the correct answer within acceptable time. However, the backward analysis of Kronos is only applicable to a single automaton. Therefore, we have to compute the product automaton beforehand and this procedure is limited by the size of the product because Kronos does not accept automata of arbitrary size.

It is possible to improve the model checking if we are able to reduce the model in a way such that we can draw conclusions from the results of the smaller model about the bigger model. To this end we define a notion of *simulation* to formalise a relation between two models.

Definition 5 (Simulation) Let $\mathcal{T}_i = (\mathcal{S}_i, \mathcal{X}_i, \mathcal{L}, \mathcal{E}_i, \mathcal{IV}_i, \mathcal{P}_i, \mu_i, S_{0,i})$ with $i = 1, 2$ Timed Automata. We say that $\mathcal{T}_1$ is a *simulation* of $\mathcal{T}_2$ (in symbols: $\mathcal{T}_1 \lesssim \mathcal{T}_2$) iff holds:

$$\begin{aligned} & \mathcal{P}_2 \subseteq \mathcal{P}_1 \\ & \text{and } \forall ((s_j^1, v_j^1, t_j)_{j \in \mathbb{N}_0}) \in \mathcal{R}(\mathcal{T}_1) : \\ & \quad \exists ((s_j^2, v_j^2, t_j)_{j \in \mathbb{N}_0}) \in \mathcal{R}(\mathcal{T}_2) : \\ & \quad \forall j \in \mathbb{N}_0 : \mu_2(s_j^2) = \mathcal{P}_2 \cap \mu_1(s_j^1) \end{aligned}$$

Roughly speaking, $\mathcal{T}_1 \lesssim \mathcal{T}_2$ holds if for each run of $\mathcal{T}_1$ there is a run of $\mathcal{T}_2$ with the same time stamps and the same propositions in $\mathcal{P}_2$.

The definition of $\lesssim$ corresponds directly to the definition of $\leq_{\infty T}^t$ by Lynch and Vaandrager. In [LV96] they defined for the timed transition systems A, B the relation $A \leq_{\infty T}^t B$ as inclusion of infinite, non-Zeno traces speaking over the visual events of A and B . In Def. 5 we conceive the propositions as visual entities and thus we require for each run of $\mathcal{T}_1$ a run of $\mathcal{T}_2$ with the same time stamps and the same observable behaviour. In the sense of [LV96] we require trace inclusion. For an extensive analysis of timed simulation relations the reader is referred to [LV96].

The definition of simulation is given on the semantical level. More convenient are methods to prove simulation on the syntactical level. To this end we spend a little work on giving a sufficient condition on the syntactical level to prove simulation.

Lemma 1 Let $\mathcal{T}_i = (\mathcal{S}_i, \mathcal{X}, \mathcal{L}, \mathcal{E}_i, \mathcal{IV}_i, \mathcal{P}_i, \mu_i, S_{0,i})$ with $i = 1, 2$ Timed Automata with $\mathcal{P}_2 \subseteq \mathcal{P}_1$. If a function $f : S_1 \mapsto S_2$ exists with

$$\begin{aligned} & f(S_{0,1}) \subseteq S_{0,2} \\ & \forall s_1 \in S_1 : \mu_1(s_1) \cap \mathcal{P}_2 = \mu_2(f(s_1)) \\ & \forall s_1 \in S_1 : \mathcal{IV}_1(s_1) \implies \mathcal{IV}_2(f(s_1)) \\ & \forall (s_1, L, \phi, R, s'_1) \in \mathcal{E}_1 : (f(s_1), L, \phi, R, f(s'_1)) \in \mathcal{E}_2 \end{aligned}$$

then $\mathcal{T}_1 \lesssim \mathcal{T}_2$ holds.

Proof. Let $r = ((s_j, v_j, t_j)_{j \in \mathbb{N}_0}) \in \mathcal{R}(\mathcal{T}_1)$ be a run of $\mathcal{T}_1$. Then it is clear that $((f(s_j), v_j, t_j)_{j \in \mathbb{N}_0})$ is a run of $\mathcal{T}_2$:

- $s_0 \in S_{0,1}$ and thus $f(s_0) \in f(S_{0,1}) \subseteq S_{0,2}$.
- $t_0 = 0, \forall x \in X : v_0(x) = 0$, and time is diverging because r is a run of $\mathcal{T}_1$.
- for all $i \in \mathbb{N}_0$ and $0 \leq t \leq t_{i+1} - t_i$ is $\mathcal{IV}_1(s_i)(v_i + t)$ fulfilled because r is a run. Hence, $\mathcal{IV}_2(f(s_i))(v_i + t)$ holds because the invariant of s_i is stronger than the invariant of $f(s_i)$.
- for all $i \in \mathbb{N}_0$ exists an edge $(f(s_i), L, \phi, R, f(s_{i+1})) \in \mathcal{E}_2$ with

$$\begin{aligned} & \phi(v_i + t_{i+1} - t_i) \\ & \text{and } v_{i+1} = (v_i + t_{i+1} - t_i)[R := 0] \end{aligned}$$

because there is an edge $(s_i, L, \phi, R, s_{i+1}) \in \mathcal{E}_1$ with these properties. Otherwise r would not be a run of $\mathcal{T}_1$.

□

In the following lemma we give a sufficient condition when it is possible to draw conclusions from model-checking results of abstracted models.

Lemma 2 (Correctness of abstractions) Let $\mathcal{T}$ and $\mathcal{T}'$ be Timed Automata with $\mathcal{T} \lesssim \mathcal{T}'$ and let φ be a TCTL-formula over $\mathcal{P}_{\mathcal{T}'}$ without use of clock constraints $x \sim c$ and $x - y \sim c$. If φ is of the form

$$p, \neg \exists \varphi_1 \mathcal{U}_{\sim c} \varphi_2, \forall \varphi_1 \mathcal{U}_{\sim c} \varphi_2$$

and $\mathcal{T}' \models \varphi$ holds, then is $\mathcal{T} \models \varphi$ valid. If φ is of the form

$$\exists \varphi_1 \mathcal{U}_{\sim c} \varphi_2, \neg \forall \varphi_1 \mathcal{U}_{\sim c} \varphi_2,$$

and $\mathcal{T}' \not\models \varphi$ is true, then $\mathcal{T} \not\models \varphi$ is true.

Proof. The assumption $\mathcal{T} \lesssim \mathcal{T}'$ assures that all runs of $\mathcal{T}$ have a corresponding run of $\mathcal{T}'$. For the first kind of TCTL-formulas we know that $\mathcal{T}' \models \varphi$ means that all runs of $\mathcal{T}'$ fulfill a condition speaking of propositions in $\mathcal{P}_{\mathcal{T}'}$. Hence, all runs of $\mathcal{T}$ have to fulfill the condition, too.

The second case can be reduced to the first case with $\neg \varphi$. $\square$

Note that the restriction to TCTL without explicit clock constraints is necessary because the notion of simulation ($\lesssim$) does not imply any relation between the clocks of the automata involved.

In the following we explain some abstraction methods which are useful in the setting of PLC-Automata. Clocks are the most expensive entities in Timed Automata when model checking is intended. Therefore, it is advantageous to use abstractions of clocks:

Definition 6 (Clock-Abstraction) Let $\mathcal{T} = (\mathcal{S}, \mathcal{X}, \mathcal{L}, \mathcal{E}, \mathcal{IV}, \mathcal{P}, \mu, S_0)$ be a Timed Automaton and $clk \in \mathcal{X}$. An automaton

$$\mathcal{T}' = (\mathcal{S}, \mathcal{X} \setminus \{clk\}, \mathcal{L}, \mathcal{E}', \mathcal{IV}', \mathcal{P}, \mu, S_0)$$

with

$$\forall (s_1, L, \phi, \rho, s_2) \in \mathcal{E} :$$

$$\exists (s_1, L, \phi', \rho \setminus \{clk\}, s_2) \in \mathcal{E}' :$$

$$\forall v \in [\mathcal{X} \mapsto \mathbf{R}_{\geq 0}] : \phi(v) \implies \phi'(v|_{\mathcal{X} \setminus \{clk\}})$$

$$\text{and } \forall s \in \mathcal{S}, v \in [\mathcal{X} \mapsto \mathbf{R}_{\geq 0}] : \mathcal{IV}(s)(v) \implies \mathcal{IV}'(s)(v|_{\mathcal{X} \setminus \{clk\}})$$

is called an *clk-abstraction* of $\mathcal{T}$.

Thus, a *clk*-abstraction of $\mathcal{T}$ can be constructed by removing *clk* from $\mathcal{X}$ and replacing all clock constraints and invariants using *clk* by weaker constraints resp. invariants not using *clk*.

Lemma 3 (Clock-Abstraction) Let $\mathcal{T}$ and clk as in Def. 6. If $\mathcal{T}'$ is a *clk*-abstraction of $\mathcal{T}$, then holds:

$$\mathcal{T} \lesssim \mathcal{T}'$$

Proof. If $((s_i, v_i, t_i)_{i \in \mathbb{N}_0})$ is a run of $\mathcal{T}$, then it is easy to verify that

$$((s_i, v_i|_{\mathcal{X} \setminus \{clk\}}, t_i)_{i \in \mathbb{N}_0})$$

is a run of $\mathcal{T}'$. $\square$

A good candidate for a clock abstraction in the TA semantics of PLC-Automata is the clock x . If we abstract these clocks in the system under consideration we can save three clocks in total because the system consists of three automata.

Another way to decrease the complexity of model checking is to reduce the number of locations of the automaton. We will use the following lemma to reduce the number of locations in the TA semantics.

Table 2. Sizes of the TA models

	locations	transitions	ints	clocks
Kronos				
Full model	1969	10132	–	7
Env. abstr.	493	1751	–	7
x abstr.	1969	10132	–	4
both	493	1751	–	4
Uppaal				
Full model	2/4/2/1/1	16/28/16/2/2	14	7
Env. abstr.	2/4/2	14/28/14	10	7
x abstr.	2/4/2/1/1	16/28/16/2/2	14	4
both	2/4/2	14/28/14	10	4

Table 3. Model-checking results

model	Uppaal	Kronos (forw.)	Kronos (backw.)
Full model	3 h 53 m	–	4 s
Env. abstr.	1 m 7 s	4 m 13 s	1 s
x abstr.	2 m 33 s	1 m 30 s	2 s
both	6 s	16 s	1 s

Lemma 4 (State reduction) Let $\mathcal{T} = (\mathcal{S}, \mathcal{X}, \mathcal{L}, \mathcal{E}, \mathcal{IV}, \mathcal{P}, \mu, S_0)$ be a Timed Automaton. Let f be a function with $\text{dom}(f) = \mathcal{S}$ and $\mathcal{P}' \subseteq \mathcal{P}$. If

$$\forall s, s' \in \mathcal{S} : f(s) = f(s') \implies \mu(s) \cap \mathcal{P}' = \mu(s') \cap \mathcal{P}'$$

holds, then is $\mathcal{T}$ a simulation of the Timed Automaton

$$\mathcal{T}_{f, \mathcal{P}'} \stackrel{\text{df}}{=} (f(\mathcal{S}), \mathcal{X}, \mathcal{L}, \mathcal{E}_f, \mathcal{IV}_f, \mathcal{P}', \mu_f, f(S_0))$$

where

$$\begin{aligned} \mathcal{E}_f &= \{(f(s), L, \phi, R, f(s')) \mid (s, L, \phi, R, s') \in \mathcal{E}\} \\ \mathcal{IV}_f(f(s)) &= \bigvee_{s' \in f^{-1}(f(s))} \mathcal{IV}(s') \\ \mu_f(f(s)) &= \mu(s) \cap \mathcal{P}' \end{aligned}$$

In symbols: $\mathcal{T} \lesssim \mathcal{T}_{f, \mathcal{P}'}$.

Note that $\mathcal{IV}_f$ and μ_f are well-defined.

Proof. Apply Lemma 1 on page 116.

For the system of Fig. 7 it is reasonable to apply Lemma 4 to the Timed Automata representing the semantics of A1 and A2. Their locations carry the information about the input of the environments Env1 resp. Env2. For the mutual exclusion property this information is not of interest. Therefore we can apply a function that projects the locations of type

$$\{0, 1, 2, 3\} \times (\mathbf{B} \times \{\mathbf{W1}, \mathbf{G1}, \mathbf{W2}, \mathbf{G2}\}) \times (\mathbf{B} \times \{\mathbf{W1}, \mathbf{G1}, \mathbf{W2}, \mathbf{G2}\}) \times \{u_i, s_i\}$$

down to locations of type

$$\{0, 1, 2, 3\} \times \{\mathbf{W1}, \mathbf{G1}, \mathbf{W2}, \mathbf{G2}\} \times \{\mathbf{W1}, \mathbf{G1}, \mathbf{W2}, \mathbf{G2}\} \times \{u_i, s_i\}.$$

In order to fulfill the property of Lemma 4 we have to remove the Envi-input proposition from the set of propositions of the new locations. In sum, we get automata A1' and A2' with the fourth of the locations as A1 and A2.

Let $\mathcal{S}$ denote the system given in Fig. 7. Now we can apply the model checkers to three abstractions:

- The system $\mathcal{S}_1$ where the Envi-inputs are abstracted.
- The system $\mathcal{S}_2$ where all x -clocks are abstracted.
- The system $\mathcal{S}_3$ where the Envi-inputs *and* the x -clocks are abstracted.

The sizes of the system are given in Table 2 while the model-checking results are given in Table 3. We can observe that the combination of both abstractions enables us to explore the whole state space within less than a minute.

Since $S \lesssim S_i \lesssim S_3$ holds with $i = 1, 2$ we can conclude by Lemma 2 from the model-checking results for S_1 , S_2 , and S_3 that also S fulfils (22).

10. Conclusion

The paper demonstrated that dense real-time model checking is becoming feasible for tiny and small systems within acceptable time. If the property that has to be shown is expressible in TCTL it seems to be a natural decision to apply the model checkers.

In case of *tiny* systems model checking is preferred since the tools will provide faster and more reliable answers.

In case of *small* systems model checking is still reasonable. However, the experiments presented indicate that the user still has to understand the system that should be verified. This is because abstractions are necessary. These abstractions usually have to be made by the user and they have to be justified. Hence, human interaction and reasoning is necessary even when model checkers will be applied. Therefore, pure logical reasoning (like using DC) is equivalent from our point of view. It requires the same: human brains and a good understanding of the problem.

In case of *larger* systems the purpose of model checkers is limited to find bugs in early design stages. It is hopeless to search the whole state space to prove the absence of bugs. Hence, logical reasoning is still the only way to prove correctness in the case of larger systems.

Acknowledgements

The author thanks E.-R. Olderog and the members of the “semantics group” in Oldenburg for fruitful discussions on the subject of this paper. Moreover, he would like to thank the anonymous referees for several helpful remarks on how to improve the paper. This research was partially supported by the German Ministry for Education and Research (BMBF), project UniForM, grant no. FKZ 01 IS 521 B3, and partially supported by the Leibniz Programme of the Deutsche Forschungsgemeinschaft (DFG) under grant No. OI 98/1-1.

References

- [ACD90] Alur R, Courcoubetis C, Dill D (1990) Model-checking for real-time systems. In: Fifth annual IEEE symposium on logic in computer science, IEEE Press, pp 414–425
- [ACD93] Alur R, Courcoubetis C, Dill D (1993) Model-checking in dense real-time. Inform Comput 104(1):2–34
- [AD90] Alur R, Dill DL (1990) Automata for modeling real-time systems. In: Paterson MS (ed) ICALP 90: automata, languages, and programming, vol 443 of lecture notes in computer science, Springer, Berlin Heidelberg New York, pp 322–335
- [AD94] Alur R, Dill DL (1994) A theory of timed automata. Theor Comput Sci 126:183–235
- [AHS96] Alur R, Henzinger TA, Sontag ED, (eds) (1996) Hybrid systems III—verification and control, vol 1066 of lecture notes in computer science. Springer, Berlin Heidelberg New York
- [BLL⁺96] Bengtsson J, Larsen KG, Larsson F, Pettersson P, Yi W Uppaal (1996) A tool suite for automatic verification of real-time systems. In: [AHS96], pp 232–243
- [DFMV98a] Dierks H, Fehnker A, Mader A, Vaandrager FW (1998) Operational and logical semantics for polling real-time systems. In: [RR98], pp 29–40
- [DFMV98b] Dierks H, Fehnker A, Mader A, Vaandrager FW (1998) Operational and logical semantics for polling real-time systems. Technical Report CSI-R9813, Computer Science Institute Nijmegen, Faculty of mathematics and informatics, Catholic University of Nijmegen
- [Die97] Dierks H (1997) PLC-Automata: a new class of implementable real-time automata. In: Bertran M, Rus T (eds) ARTS’97, vol 1231 of lecture notes in computer science, Mallorca, Spain, Springer, Berlin Heidelberg New York, pp 111–125
- [Die99] Dierks H (1999) Specification and verification of polling real-time systems. PhD Thesis, University of Oldenburg
- [DOTY96] Daws C, Olivero A, Tripakis S, Yovine S (1996) The tool kronos. In: [AHS96], pp 208–219
- [DT98] Dierks H, Tapken J (1998) Tool-supported hierarchical design of distributed real-time systems. In: Proceedings of the 10th euromicro workshop on real time systems, IEEE computer society, pp 222–229
- [HNSY94] Henzinger T, Nicollin X, Sifakis J, Yovine S (1994) Symbolic model checking for real-time systems. Inform Comput 111:193–244
- [HZ97] Hansen MR, Zhou C (1997) Duration calculus: logical foundations. Formal Aspects Comput 9:283–330
- [KBPO⁺96] Krieg-Brückner B, Peleska J, Olderog E-R, Balzer D, Baer A (1996) UniForM—universal formal methods workbench. In: Grote U, Wolf G (eds) Statusseminar des BMBF Softwaretechnologie, BMBF, Berlin, pp 357–378
- [LPW97] Larsen KG, Pettersson P, Yi W (1997) Uppaal in a nutshell. Int J Softw Tools Tech Trans 1(1-2):134–152
- [LV96] Lynch N, Vaandrager FW (1996) Forward and backward simulations part II: timing-based systems. Inform Comput 128(1):1–25

- [Mos85] Moszkowski B (1985) A temporal logic for multilevel reasoning about hardware. *IEEE Comput* 18(2):10–19
- [MP95] Maler O, Pnueli A (1995) Timing analysis of asynchronous circuits using timed automata. In: *Proceedings CHARME'95*, vol 987 of *lecture notes in computer science*, Springer, Berlin Heidelberg New York, pp 189–205
- [MY96] Maler O, Yovine S (1996) Hardware timing verification using kronos. In: *Proceedings 7th conference on computer-based systems and software engineering*. IEEE Press
- [NSY92] Nicollin X, Sifakis J, Yovine S (1992) Compiling real-time specifications into extended automata. *IEEE Trans Software Eng* 18(9):794–804
- [RR98] Ravn AP, Rischel H (eds) (1998) *Formal techniques in real-time and fault-tolerant systems*, vol 1486 of *lecture notes in computer science*, Lyngby, Denmark, Springer, Berlin Heidelberg New York
- [TD98] Tapken J, Dierks H (1998) MOBY/PLC – graphical development of PLC-automata. In: [RR98], pp 311–314
- [Yov97] Yovine S (1997) Kronos: a verification tool for real-time systems. *Int J Softw Tools Tech Trans* 1(1-2):123–133
- [Zho93] Zhou C (1993) Duration calculi: An overview. In: Bjørner D, Broy M, Pottosin IV (eds) *Formal methods in programming and their application*, vol 735 of *lecture notes in computer science*, Springer, Berlin Heidelberg New York, pp 256–266
- [ZHR91] Zhou C, Hoare CAR, Ravn AP (1991) A calculus of durations. *Inform Proc Lett* 40(5):269–276

Received June 1999

Accepted in revised form September 2003 by M. R. Hansen and C. B. Jones

Published online 28 April 2004